

1. Introduction to Graphs

A graph is a data structure used to represent relationships between objects.

A graph contains:

- **Vertices (Nodes)** → represent objects
- **Edges (Links)** → represent connections

Mathematically:

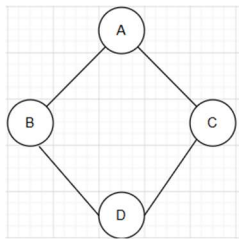
$$G = (V, E)$$

Where:

- V = Vertices
- E = Edges

2. Basic Graph Visualization

Example Graph



Explanation

Vertices:

- A, B, C, D

Edges:

- A-B
- A-C
- B-C
- B-D
- C-D

3. Graph Terminologies with Visual Examples

3.1 Vertex (Node)

A vertex is a point in the graph.

(A)

Here:

- A is a vertex.

3.2 Edge

An edge connects two vertices.

A ----- B

This means:

- A is connected to B.

3.3 Directed Graph (Digraph)

Edges have directions.

A -----> B

Meaning:

- A points to B
- B may not point to A

Real-Life Example

Instagram Follow System

Alice -----> Bob

Alice follows Bob.

3.4 Undirected Graph

Edges have no direction.

A ----- B

Both vertices connect equally.

Real-Life Example

Facebook Friendship

Alice ----- Bob

3.5 Weighted Graph

Edges contain weights.

5
A ----- B

Weight may represent:

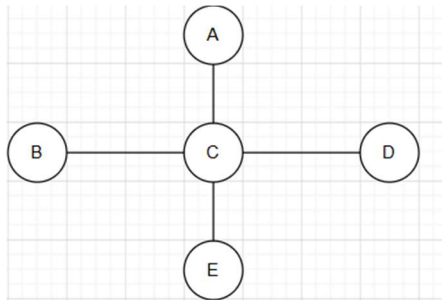
- Distance
- Cost
- Time
- Etc

Real-Life Example (Road Network)

Kigali ----80km---- Musanze

3.6 Degree of a Vertex

The number of edges connected to a vertex.



Degree of C:

$$\text{degree}(C) = 4$$

3.7 Path

A sequence of connected vertices.

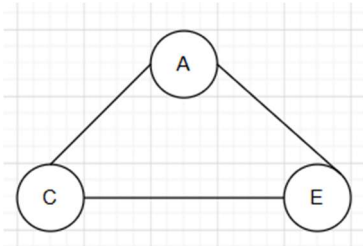
A ---- B ---- C ---- D

Path:

- $A \rightarrow B \rightarrow C \rightarrow D$

3.8 Cycle

A cycle starts and ends at the same vertex.



Cycle:

- $A \rightarrow B \rightarrow C \rightarrow A$

3.9 Connected Graph

Every vertex can be reached.

$A \text{ ---- } B \text{ ---- } C \text{ ---- } D$

All vertices are connected.

3.10 Disconnected Graph

Some vertices cannot be reached.

$A \text{ ---- } B \quad C \text{ ---- } D$

Two separate groups exist.

4. Graph Representations

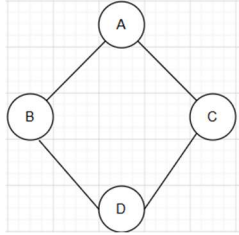
Graphs are stored using:

1. Adjacency Matrix
2. Adjacency List

5. Adjacency Matrix

Uses a 2D array.

Visual Graph



Matrix Representation

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Matrix Interpretation

Example:

$$\text{Matrix}[A][B] = 1$$

Meaning:

- A connects to B

6. Adjacency List

Stores neighbors using linked lists.

Visual Representation

A → B → C

B → A → D

C → A → D

D → B → C

7. Breadth-First Search (BFS)

Introduction to BFS

Breadth-First Search (BFS) is a graph traversal algorithm used to explore vertices systematically level by level.

Instead of going deeply into one path immediately, BFS first visits:

1. The starting vertex
2. All neighboring vertices
3. Then neighbors of those neighbors
4. Continues outward gradually

BFS is one of the most important graph algorithms because it guarantees the shortest path in an unweighted graph.

Real-Life Analogy of BFS

Imagine dropping a stone into water.

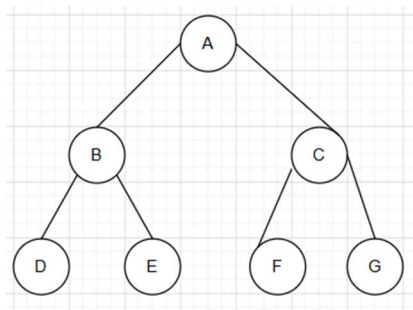
The waves spread outward evenly in all directions.

BFS behaves in the same way:

- It expands outward layer by layer from the starting vertex.

BFS Visualization

Consider the graph below:



Starting traversal from vertex **A**.

BFS Traversal Process

Step 1 — Visit A

Visited: A

Queue:

B, C

BFS first visits A, then places all neighbors of A into the queue.

Step 2 — Visit B

Visited: A, B

Queue:

C, D, E

B is removed from the queue and visited.

Its neighbors D and E are added.

Step 3 — Visit C

Visited: A, B, C

Queue:

D, E, F, G

C is visited.

Its neighbors F and G are added.

Step 4 — Continue Level-by-Level

Visited Order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Important Characteristics of BFS

Characteristic	Description
Traversal Style	Level-by-level
Data Structure Used	Queue (FIFO)
Revisiting Prevention	Visited Array
Shortest Path	Guaranteed in unweighted graphs

Memory Usage	Higher than DFS
--------------	-----------------

Why BFS Uses a Queue

BFS uses a **Queue** because queues follow:

FIFO = First In First Out

This means:

- The first discovered vertex is explored first.

This perfectly supports level-by-level traversal.

BFS Algorithm Logic

1. Start from a selected vertex.
2. Mark it as visited.
3. Insert it into the queue.
4. Remove a vertex from the queue.
5. Visit all unvisited neighbors.
6. Add neighbors to the queue.
7. Repeat until the queue becomes empty.

BFS Pseudocode

```
BFS(Graph, start):  
  
    create visited array  
  
    mark start as visited  
  
    create queue  
  
    enqueue(start)  
  
    while queue is not empty:  
  
        vertex = dequeue()  
  
        print(vertex)  
  
        for each neighbor of vertex:  
  
            if neighbor is not visited:  
  
                mark neighbor visited  
  
                enqueue(neighbor)
```


BFS in C (Improved Version)

```
#include <stdio.h>

#define SIZE 7

int queue[SIZE];

int front = -1;

int rear = -1;

/* Add element to queue */
void enqueue(int value)
{
    if(front == -1)
        front = 0;
    queue[++rear] = value;
}

/* Remove element from queue */
int dequeue()
{
    return queue[front++];
}

/* BFS Traversal */
void BFS(int graph[SIZE][SIZE], int start)
{
    int visited[SIZE] = {0};
    visited[start] = 1;
    enqueue(start);
    while(front <= rear)
    {
```

```

int vertex = dequeue();

printf("%d ", vertex);

for(int i = 0; i < SIZE; i++)
{
    if(graph[vertex][i] == 1 && !visited[i])
    {
        visited[i] = 1;
        enqueue(i);
    }
}
}

int main()
{
    int graph[SIZE][SIZE] = {
        {0,1,1,0,0,0,0},
        {1,0,0,1,1,0,0},
        {1,0,0,0,0,1,1},
        {0,1,0,0,0,0,0},
        {0,1,0,0,0,0,0},
        {0,0,1,0,0,0,0},
        {0,0,1,0,0,0,0}
    };

    printf("BFS Traversal:\n");

    BFS(graph, 0);

```

```
    return 0;  
}
```

Applications of BFS

1. Shortest Path Algorithms

Used in:

- GPS systems
- Maps
- Network routing

2. Social Networks

Finding:

- Mutual friends
- Suggested connections

3. Web Crawlers

Search engines explore websites level by level.

4. Network Broadcasting

Used in:

- Computer networks
- Broadcasting systems

Time Complexity of BFS

$$O(V + E)$$

Where:

- V = Number of vertices
- E = Number of edges

Reason:

- Every vertex is visited once.
- Every edge is checked once.

8. Depth-First Search (DFS)

Introduction to DFS

Depth-First Search (DFS) is a graph traversal algorithm that explores a path as deeply as possible before backtracking.

Unlike BFS, DFS does not explore neighbors level-by-level.

Instead:

1. It chooses one path
2. Travels deeply along that path
3. Backtracks when stuck
4. Continues exploring unexplored paths

DFS is highly recursive in nature.

Real-Life Analogy of DFS

Imagine exploring a maze.

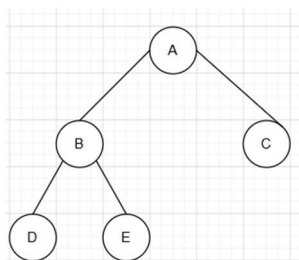
You keep moving forward
until you hit a dead end.

Then you backtrack
and try another path.

That is exactly how DFS works.

DFS Visualization

Consider the graph below:



Starting traversal from vertex **A**.

DFS Traversal Process

Step 1 — Visit A

Visited:

A

DFS chooses one branch.

Step 2 — Go Deep to B

Visited:

A → B

Step 3 — Continue Deep to D

Visited:

A → B → D

D has no unexplored neighbors.

DFS now backtracks.

Step 4 — Backtrack to B and Visit E

Visited:

A → B → D → E

Step 5 — Backtrack to A and Visit C

Visited:

A → B → D → E → C

Final DFS Traversal

A → B → D → E → C

Important Characteristics of DFS

Characteristic	Description
Traversal Style	Deep exploration
Data Structure Used	Stack / Recursion
Backtracking	Yes
Memory Usage	Lower than BFS
Shortest Path Guarantee	No

Why DFS Uses Recursion or Stack

DFS naturally behaves like a stack.

A stack follows:

LIFO = Last In First Out

Meaning:

- The most recently discovered vertex is explored first.

This supports deep exploration.

DFS Algorithm Logic

1. Start from a selected vertex.
2. Mark it visited.
3. Visit one unvisited neighbor.
4. Continue deeply.
5. Backtrack when no neighbors remain.
6. Continue until all vertices are explored.

DFS Pseudocode

```
DFS(vertex):  
  
    mark vertex visited  
  
    print(vertex)  
  
    for each neighbor:  
  
        if neighbor not visited:  
  
            DFS(neighbor)
```

DFS Recursive Implementation in C

```
#include <stdio.h>

#define SIZE 5

void DFS(int graph[SIZE][SIZE], int vertex, int visited[])
{
    visited[vertex] = 1;

    printf("%d ", vertex);

    for(int i = 0; i < SIZE; i++)
    {
        if(graph[vertex][i] == 1 && !visited[i])
        {
            DFS(graph, i, visited);
        }
    }
}

int main()
{
    int visited[SIZE] = {0};

    int graph[SIZE][SIZE] = {
        {0,1,1,0,0},
        {1,0,0,1,1},
        {1,0,0,0,0},
        {0,1,0,0,0},
        {0,1,0,0,0}
    };
};
```

```
printf("DFS Traversal:\n");  
DFS(graph, 0, visited);  
return 0;  
}
```

Understanding Recursion in DFS

When DFS calls itself repeatedly:

```
DFS(A)  
  DFS(B)  
    DFS(D)
```

The system stores unfinished calls in memory using the call stack.

When D finishes:

- The algorithm returns to B
- Then continues exploring

This process is called:

- Backtracking

Applications of DFS

1. Cycle Detection

Determines whether loops exist in graphs.

2. Maze Solving

Used in:

- Robotics
- Games
- Navigation systems

3. Topological Sorting

Used in:

- Task scheduling
- Compiler dependency analysis

4. Connected Components

Used in:

- Social networks
- Image processing

Time Complexity of DFS

$$O(V + E)$$

Where:

- V = Vertices
- E = Edges

Reason:

- Every vertex is visited once.
- Every edge is explored once.

BFS vs DFS Summary

Feature	BFS	DFS
Traversal	Level-by-level	Deep exploration
Structure Used	Queue	Stack/Recursion
Shortest Path	Yes	No
Memory Usage	Higher	Lower
Best Use Cases	Shortest paths	Backtracking & recursion
Traversal Nature	Wide	Deep

References

1. Sedgewick, R. (1990). *Algorithms in C*.
2. Weiss, M. A. (1993). *Data structures and algorithm analysis in C*.

Expectations:

1. Think of a computational problem
2. The problem can be solved using graph
3. Implement the pseudo code for the solution
4. Implement the solution in C
5. Test the solution, deploy it on linux environment